OdinANN: Direct Insert for Consistently Stable Performance in Billion-Scale Graph-Based Vector Search

Hao Guo Youyou Lu*

Tsinghua University

Abstract

Approximate Nearest Neighbor Search (ANNS) is widely used in various scenarios. For billion-scale ANNS, on-disk graph-based indexes, which organize the vectors as a graph and store them on disk, are favored for their performance and cost-efficiency. However, existing indexes can not maintain a stable search performance while inserting new vectors.

In this paper, we propose to use *direct insert*, which directly inserts vectors into the on-disk index, rather than buffering them in memory and merging them to disk in batches like existing systems. This approach can even out the interference of insert with frontend search, thus stabilizing the performance. We evaluate direct insert by integrating it into a billion-scale graph-based ANNS index named OdinANN. With a fixed insert rate, OdinANN outperforms state-of-the-art ANNS indexes in search latency and throughput, and it consistently shows stable performance in billion-scale vector datasets.

1 Introduction

Approximate nearest neighbor search (ANNS) is the key to multi-modal data retrieval in web search [10, 17] and retrieval-augmented generation (RAG) [15]. Multi-modal data, such as texts and images, are encoded to high-dimensional vectors using neural networks [9, 19]. An ANNS index is built upon these vectors and traversed during a search to get the k nearest neighbors of the query vector. Among all types of ANNS indexes, *on-disk graph-based indexes* [24, 27], in which vectors are organized as a graph and stored on disk, are favored for their performance and cost-efficiency to support large-scale vector search (e.g., billions of vectors [7, 22, 29]).

There is a strong need for ANNS indexes to support vector updates because current systems generate new data continuously [16, 26, 29, 30]. Vector updates can be handled in two ways, index rebuild and index update. Index rebuild, which takes several days in billion-scale datasets [30], fails to keep search results up-to-date. Therefore, index update is favored by recent on-disk graph-based indexes [23]. They support vector inserts and deletes using *buffered insert and delete*, where they absorb updates into an in-memory index, and then bulk merge the in-memory index into the on-disk index periodically. This way reduces the overhead of index updates by combining disk writes during merge, thus enabling real-time vector search and online index update.

However, by experimental analysis, we find that buffered insert is inefficient when merging inserts to disk. First, merge interferes with frontend search tasks (e.g., increases their median latency to 1.54×), owing to disk bandwidth interference. Second, merge has a high memory consumption (e.g., 125GB for merging 3% of vectors into a billion-scale index [23]), which consists of the in-memory index for inserts and buffered disk writes during the merge. Third, merge gains little performance boost even with a large insert batch, whose throughput is capped at 3000 QPS in our experiment. This is because merge first finds the neighbors for the inserted vectors using on-disk ANNS. This process can not be batched efficiently and thus is executed one by one.

In this paper, we propose to use *direct insert* for on-disk graph-based ANNS indexes, where vectors are directly inserted into the on-disk index one by one, rather than buffered in memory and merged to disk later as in buffered insert. This approach can even out the insert cost and avoid the in-memory index used by buffered insert, thus stabilizing the frontend search performance and saving memory. Also, it is possible to make direct insert as fast as buffered insert, because buffered insert does not benefit much from batching. However, it is non-trivial to achieve efficient direct insert in graph-based indexes. There are several challenges as follows.

(1) High disk write overhead of direct insert. Direct insert updates tens or hundreds of neighbor records of the target vector. Updating them in place incurs massive disk writes. To combine these record updates for fewer disk writes, a straightforward way is to use a log-structured data layout, but it suffers from garbage collection (GC) overhead.

(2) Complex concurrency control with search. Direct insert searches the target vector to find its neighbors, which

*Youyou Lu is the corresponding author (luyouyou@tsinghua.edu.cn).